scientific reports

Check for updates

OPEN

Graph embedding based label propagation for community detection in social networks

Shyam Sundar Meena[1], Prakash Chandra Sharma[2]✉, Yadvendra Pratap Singh[3]✉ & Mukund Pratap Singh[4]

Community structures are common features of many real-world networks, and community detection is necessary to understand how these networks are organized. Various approaches have been devised for community detection, with each providing varying degrees of both accuracy and structural understanding. One of them, the Label Propagation Algorithm, is so common because it is simple and computationally cheap. Nevertheless, it does not usually reach great modularity and yields inaccurate community counts and structures in real-world networks. This is mostly due to its naive criteria of selecting the neighbor nodes when it comes to label propagation. To tackle the issue, we developed an adjusted algorithm, which we call Embedding-based Label Propagation (ELP), a hybrid between LPA and node embedding that allows us to combine both local connectivity and global structural data. ELP update step takes into consideration not only the local neighborhood, as in conventional LPA, but also embedding-based similarities to inform more productive neighbor selection. We tested ELP on popular benchmark datasets such as Karate Club, Dolphins, Football, Polbooks, and LFR synthetic networks and compared its results with LPA and other well-established algorithms. The empirical findings show that ELP can always perform better in modularity, NMI and NF1 scores, but it is also scalable to large and complex networks. These results can be used to identify ELP as an effective and powerful method of community-finding in real and artificial-world scenarios.

Keywords Label propagation algorithm, Social network, Community detection, Graph embedding, Modularity

In the real world, data of various domains follow complex network structures that can be modelled mathematically as a graphical structure, where nodes and links between them represent objects and relationships. Example of such networks are social networks[1], protein-protein networks[2], and citation networks[3]. One of the characteristics of complex networks is the community structure[4–6] Such structures are characterized by a high degree of inter connectivity within community while having inter few interrelationships with nodes outside of the community. The community in literature also referred to as clusters or modules[7,8]. All in all, knowledge of community structures enhances our understanding of social and other networks. Knowing community structure can assist academics, scientists, and even ordinary people see connectivity patterns and engagements in these networks.

Several techniques have already been presented in the scientific literature to address group detecting issues. One important algorithm is the Label Propagation Algorithm (LPA). LPA is widely used since it requires minimal execution duration and is easy. But it falls short in precision within regards to modularity, and the total amount of communities discovered per count, and the organization of communities in actual networks. This problem arises because neighboring node picking rules in the label propagation mechanism are poorly designed. Several advanced algorithms have been suggested to address current LPA[9] issues, including[10–14]. The most recent algorithm is LBLD (Local Balanced Label Diffusion)[10]. The LBLD algorithm tries to make forming communities faster by looking at both inside and outside views. However, it has some issues. First, its way of updating labels only focuses on important nodes. This means it attracts too many other nodes to those communities, which can spoil the quality of the results. Second, it starts by grouping a central node, its closest neighbor, and its shared neighbors.

[1]Shri Vaishnav Vidyapeeth Vishwavidyalaya, Ujjain Road, Indore, Madhya Pradesh 453111, India. [2]Department of Information Technology, Manipal University Jaipur, Jaipur, Rajasthan 303007, India. [3]Department of Artificial Intelligence & Machine Learning, Manipal University Jaipur, Jaipur, Rajasthan 303007, India. [4]School of Computer Science Engineering and Technology, Bennett University, Greater Noida, Uttar Pradesh 201310, India. ✉email: prakashsharma12@gmail.com; yadvendra.singh@jaipur.manipal.edu; yadvendra.mnnit@gmail.com

Our proposed new approach is em-bedding-based label propagation (ELP) in order to discover communities in complex networks correctly. The proposed algorithm rearranges the LPA with the nodes embeddings, so that it can draw both global- and local information intertwined in a more optimal way, especially the big and complex networks. The ELP algorithm update step looks at the local neighborhood (like the traditional LPA) and the similarities between the node embeddings for neighbor node selection.

The article will proceed as follows: In the next section, we analyze the pertinent literature on the theory of community detection. our algorithm's main idea and the entire methodology is in section 3. Section 4 describes the results of testing the algorithm on real and synthetic networks. Section 5 has a conclusion.

Related work

In order to enhance the efficacy of the LPA [9] in community discovery activities, certain academicians have suggested modifications. The suggested approaches are developed to solve unique problems that existed in the initial LPA, including inefficiency, unpredictability and instability.

Zhang et al.[15] developed a method for detecting communities within networks and devised a way of measuring a node's importance. This approach looks at two things: the k-shell value and the node's number of connections. They also developed something called label influence, which improves the effectiveness of label distribution and determines how a label impacts nearby nodes. Using this strategy, they chose the label that made the most difference rather than randomly selecting one when there were numerous options. This made the outcomes more consistent.

The article[16] talks about a three-step process focusing on ranking nodes. This method looks at both local and big-picture network features. First, nodes are ranked by how important they are. They use a special index that checks how well a node connects with nearby nodes and its place in the larger network. In the second step, they focus on the top nodes, which they call seed nodes. Then in the last step, they use a local growth strategy. This means they keep adding nearby nodes to form a tight-knit community based on connections and closeness.

Kong and colleagues designed a technique that considers both the node's relevance and the label's impact[17]. Furthermore, they developed a unique tightening mechanism that may also be utilised for label propagation. An approach was devised by Yue et al.[18] to amend the labels of nodes to enhance the procedure's efficiency. Using the K-shell methodology, Deng et al.[19] enhanced the node importance model. Zarezade and his colleagues proposed a methodology for ascertaining node labels in their research. This methodology entailed a combination of heterogeneous node scoring and where the border nodes and labels would be updated simultaneously[20].

An approach known as Skip-LPA was presented by Thakare and colleagues in their 2016 publication [21]. Due to the fact that the technique only initialised a limited portion of the nodes responsible for label propagation, the total number of iterations performed by the algorithm was drastically decreased. The initial creators of the CDIC algorithm were Yuan and his collaborators[22]. The techniques of the algorithm provided many ways to solve the problem, one of them being the impact of the core nodes on the diffusion of labels given to the nodes. Also, the algorithm merged nodes by using the interests of the community of a given set of nodes. This made the technique feasible. Li et al.[23] and Lin et al.[24] were both concerned with stabilising their algorithmic results, the former through the use of the modularity function, and the latter with community core initialisation. It was these very issues that proved the motivation for dealing with instability.

Zhao *et al.*[25] did extensive community detection in order to reduce the temporal complexity of the technique. This was achieved through graph compression and label propagation. It was done to enhance the overall effectiveness of the algorithm.In their study, Roghani et al.[10] proposed a technique of label propagation which uses local similarity metric to weigh the importance of different nodes. The technique of Bouyer et al. devote to the growth of the low degree node sclustering community[14]. Zhang et al.[26] and Zhai et al.[27] described techniques with with and without core nodes and sequential label propagation carried out by the core nodes.

The paper [28] discusses the problem of community detection in the incomplete networks, in which the presence of missing edges due to privacy, security, or other limitations results in poor performance of both overall and deep learning-based approaches. Existing methods, including graph partitioning, modularity optimization, and link prediction, are inappropriate in these contexts- link prediction, though can be useful at recovering edges, is not always useful at improving community detection, and many deep models require node attributes or labels that are not available in real-world networks. The authors suggest the Deep Core Node Information Embedding (DCNIE) algorithm that combines edge augmentation to maintain topology, k-core decomposition to obtain dense sub-network information, a deep unsupervised embedding model with a parametrization layer to learn features efficiently, and Gaussian mixture clustering to partition them accurately in order to overcome these limitations. In contrast to previous efforts that make much use of edge recovery, DCNIE focuses more on core node data to generate robust and stable community detection. Experimental validation on eleven real world datasets with eight evaluation measures, shows that DCNIE is always superior to eleven state-of-the-art algorithms in accuracy and efficiency, providing a more robust community detention in network with missing edges.

Shen, Cencheng et al. [29] suggests a Principal Community Encoder Embedding (P-GEE) algorithm that jointly identifies principal communities and produces vertex embeddings by prioritizing the significance of a community using a new sample community score. The dimensionality reduction and enhanced interpretability and computational efficiency of the method, by only retaining the dimensions of the largest communities, preserves conditional density under the random Bernoulli graph model to guarantee Bayes-optimal classification of vertices. The reason is supported by a large body of theoretical study and evidence of the usefulness of the approach, and simulations and real-world experiments confirm the benefits of the method in incorporating visualization, ground-truth community detection, classification accuracy, and resistance to label noise and redundant communities. P-GEE is less opaque, more scalable, and robust, relative to current graph embedding models, such as node2vec, spectral, and graph neural networks, due to a non-iterative architecture and its lack of

noisy dimensions. Despite its high new directions of the study are the expansion of P-GEE to support multiple graphs, overlapping communities, and multivariate node attributes, or to transform it into iterative neural network structures to operate more efficiently with increasingly complex graph structures.

Research examines how nodes degrees as well as edges mass affect label-updating. On the other hand, the impact of both local and global significance is also quite important in networks of friends, including a social network that is comprised of little sites. The Embedding-based Label Propagation (ELP) technique that has been presented combines the positive aspects of the LPA with node embedding approaches. This allows for the synthesis of both local and global information about the network, which ultimately improves efficiency in complicated and large networks.

Proposed work

The goal of community detection is to find dense sub-clusters or modules within a network. A network can be represented as a graph[30]. In a graph, $G(V, E)$, with V vertices and E edges, community detection partition $G(V, E)$ into i clusters, where $C = \{c_1, c_2, c_3, \ldots, c_i\}$ is a collection of communities, where each $c_i \subseteq V(G)$ represents a set of nodes referred to as a community.

Community detection approach

The new approach was created with the idea of splitting it into 4 main tasks. These tasks are Graph Embedding, Distance Calculation with Embeddings, Label Propagation with Embeddings, and Community Detection. The descriptions of these tasks are given as follows:

1. **Graph Embedding:** as a first step, a graph embedding method is employed to obtain node embeddings. Specifically, we apply Node2Vec[31], an unsupervised learning approach that captures both local and global structural patterns in the network without relying on labeled data. This addition makes the embedding process and the overall learning procedure more transparent. Graph embedding uses the process of mapping the nodes i.e. V of a graph i.e. $G = (V, E)$ into a low dimensional space i.e. $\mathbb{R}^d$. Vectors in $\mathbb{R}^d$ represent node $u \in V$, and they are called embeddings $\vec{z}_u$ and preserve the structural relationships of the graph.

In Node2Vec, the goal is to learn embeddings that encode Local relationships (Connectivity of immediate neighbors) and Global structural properties (Patterns such as communities). Node2Vec uses biased random walks to capture graph structure. From a node u, the probability of transitioning to a neighboring node v_x depends on the previous node v_{t-1} and the current node v_t.

$$P(v_{t+1} = v_x \mid v_t, v_{t-1}) = \frac{\pi(v_{t-1}, v_x)}{\sum_{v' \in \mathcal{N}(v_t)} \pi(v_{t-1}, v')} \tag{1}$$

where: $\mathcal{N}(v_t)$: Set of neighbors of v_t and $\pi(v_{t-1}, v_x)$: Transition weight based on the bias.
The bias term $\pi(v_{t-1}, v_x)$ is given by:

$$\pi(v_{t-1}, v_x) = \begin{cases} \frac{1}{p}, & \text{if } d(v_{t-1}, v_x) = 0, \\ 1, & \text{if } d(v_{t-1}, v_x) = 1, \\ \frac{1}{q}, & \text{if } d(v_{t-1}, v_x) = 2. \end{cases} \tag{2}$$

Here: p: Return parameter (encourages revisiting the last node if $p > 1$), q: In-out parameter (encourages exploration if $q > 1$) and $d(v_{t-1}, v_x)$: Shortest path distance.
Sequences of nodes were generated by Random walks, which are treated like sentences. The Skip-Gram model is utilized to understand embeddings by maximizing the likelihood of context nodes v given the central node u.

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in \mathcal{N}(u)} \log P(v \mid u) \tag{3}$$

where $\mathcal{N}(u)$: Context nodes of u and $P(v \mid u)$: Conditional probability of v appearing in the context of u. The probability is modeled as:

$$P(v \mid u) = \frac{\exp(\vec{z}_u \cdot \vec{z}_v)}{\sum_{w \in V} \exp(\vec{z}_u \cdot \vec{z}_w)} \tag{4}$$

Here, $\vec{z}_u$: Embedding of node u, $\vec{z}_v$: Embedding of node v and $\vec{z}_u \cdot \vec{z}_v$: Dot product.
Consider a small social network graph given in Fig. 1 with the A, B, C, D, E nodes, in which A is connected to $B, C, D, E$and B and C are also connected.
From node A, two random walk sequences might be:

Input: $G(V, E)$
Output: Communities
Step 1: Load a Graph
$G \leftarrow$ `graph()`
Step 2: Produce Node Embeddings using node2vec
n2v $\leftarrow$ Node2Vec(G, dimensions, walk_length, num_walks, workers)
model $\leftarrow$ n2v.fit(window, min_count, batch_words)
node_embeddings $\leftarrow$ array of embeddings for each node in G
Step 3: Similarity Function using Node Embeddings
embedding_similarity $\leftarrow$ cosine_similarity(node_embeddings)
Step 4: Initialize Labels
labels $\leftarrow$ {node : node | node in G.nodes()}
Step 5: Label Propagation
FUNCTION(propagate_labels$G$, labels, embedding_similarity, max_iter $= 100$)
nodes $\leftarrow$ list of nodes in G
for $it = 1$ **to** max_iter **do**
  Shuffle nodes randomly
  updated_labels $\leftarrow$ labels.copy()
  for each node in nodes **do**
    neighbors $\leftarrow$ list of neighbors of node
    if neighbors is empty **then**
      continue
    end if
    neighbor_labels $\leftarrow$ [labels[neighbor] for neighbor in neighbors]
    node_idx $\leftarrow$ index of node in G.nodes()
    neighbor_indices $\leftarrow$ indices of neighbors in G.nodes()
    similarities $\leftarrow$ embedding_similarity[node_idx, neighbor_indices]
    label_weights $\leftarrow$ empty dictionary
    for each (neighbor_label, similarity) in zip(neighbor_labels, similarities) **do**
      if neighbor_label in label_weights **then**
        label_weights[neighbor_label] $\leftarrow$ label_weights[neighbor_label] + similarity
      else
        label_weights[neighbor_label] $\leftarrow$ similarity
      end if
    end for
    best_label $\leftarrow$ label with the highest value in label_weights
    updated_labels[node] $\leftarrow$ best_label
  end for
  if updated_labels = labels **then**
    Print "Converged after" *it* "iterations."
    Break
  end if
  labels $\leftarrow$ updated_labels
end for
return labels
ENDFUNCTION
Step 6: Run ELP algorithm
final_labels $\leftarrow$ propagate_labels(G, labels, embedding_similarity)
Step 7: Visualize the Resulting Communities
colors $\leftarrow$ [final_labels[node] for node in G.nodes()]
Draw G with node_color = colors and labels
Print “Detected Communities:", final_labels

Algorithm 1. Embedding-based Label Propagation (ELP) Algorithm

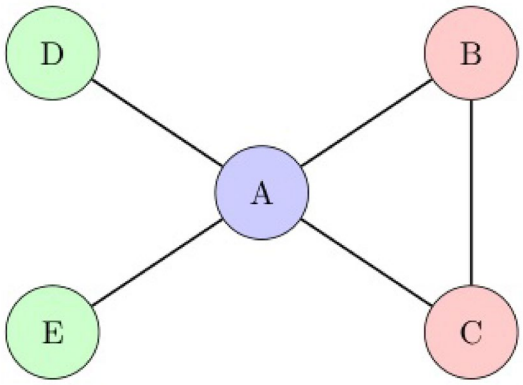


Fig. 1. An example graph.

$$A \to B \to C \to A \to D, \quad A \to C \to B \to A \to E.$$

The output embeddings are represented as:

$$Z = \begin{bmatrix} \vec{z}_A \\ \vec{z}_B \\ \vec{z}_C \\ \vec{z}_D \\ \vec{z}_E \end{bmatrix}, \quad \vec{z}_i \in \mathbb{R}^d \tag{5}$$

These embeddings capture the graph's structural properties:

- Nodes A, B, C have similar embeddings due to their dense connections.
- Nodes D, E are distinct, reflecting their peripheral roles.

2. **Similarity Calculation with Embeddings:** Using the node embeddings, similarity between nodes is calculated. This helps incorporate global graph structure when updating labels in the LPA.

Assume that the graph $G = (V, E)$ is composed of nodes $V = \{1, 2, \ldots, |V|$ and edges E. A dimensional embedding vector $\vec{z}_u \in \mathbb{R}^d$ is used to denote every node $u \in V$, where d serves as the element that represents the embedding dimension. A matrix of embeddings Z that contains each of the embeddings for nodes is as follows:

$$Z = \begin{bmatrix} \vec{z}_1 \\ \vec{z}_2 \\ \vdots \\ \vec{z}_{|V|} \end{bmatrix} \in \mathbb{R}^{|V| \times d} \tag{6}$$

The cosine similarity, S(u, v), is used to calculate the degree to which these two seem. The calculation that follows is used to determine the cosine similarity among vertices u and v.

$$S(u, v) = \frac{\vec{z}_u \cdot \vec{z}_v}{\|\vec{z}_u\| \, \|\vec{z}_v\|}, \quad \forall u, v \in V \tag{7}$$

where $\|\vec{z}_u\|$ is the Euclidean norm (magnitude) of the embedding $\vec{z}_u$.

3. **Label Propagation with Embeddings:** When nodes propagate labels, they consider both the labels of their immediate neighbors and their global similarity based on the embeddings.

The proposed approach updates the label of a node u by considering the labels of its neighbors $v \in \mathcal{N}(u)$, weighted by their similarity. The updated label $L_t(u)$ for node u at iteration t is given by:

$$L_t(u) = \arg\max_k \sum_{v \in \mathcal{N}(u)} S(u,v) \cdot \delta\big(L_{t-1}(v), k\big) \tag{8}$$

where, k indexes over all possible labels, $\delta(L_{t-1}(v), k) = 1$ if the label $L_{t-1}(v) = k$, and 0 otherwise and $L_{t-1}(v)$ is the label of node v at the previous iteration.

The similarity measure $S(u, v)$ controls the weight of each neighboring node v in determining the label of node u.

Such a mechanism implements the global graph structure in terms of embeddings and guarantees label propagation.

The propagation procedure of the label is then repeated until a convergence is found. Convergence Convergence occurs when the labels have stabilised, i.e. when $L_t(u) = L_{t-1}(u)$ for all nodes $u \in V$, or after some fixed number of iterations T.

4. **Community Detection:** After the stabilization of the propagation process, the last labels are obtained to detect communities.

Based on the above approach, a algorithm 1 is developed for the community detection and it is named as a Embedding based Label Propagation (ELP) Algorithm . ELP algorithm is an attempt to merge the label propagation approach with the node embedding-based information to identify communities of the graph. It begins by loading a graph, $G(V, E)$ which consists of nodes (V) and edges (E).

Once loaded, the algorithm uses the Node2Vec algorithm to generate a node embedding. These results represent an embedding of the nodes in terms of random walks and skip-gram model. Setting such as embeddings size (64), walk length (30), walks (200) and threads (4) can be modified to make the warm up of embeddings faster.

Next, the algorithm calculates cosine similarity to create a similarity matrix. This matrix measures how close nodes are to each other. Each node starts with its own label, usually its index number. The Hybrid Label Propagation process uses the similarity matrix to weigh how much influence nearby nodes have. In each step, nodes are selected randomly.They use labels of their neighbors, as well as the similarity values. The latest labels are made using the most powerful votes. This is carried out until the process converges or a maximum number of 100 iterations is attained.

Upon completion of this procedure, this assigns community identities to all the vertices, indicating their respective group affiliations. The outcomes can be depicted through colouring nodes according to the associated groups, facilitating the observation of the organization of the graph. This combination of techniques integrates label propagation with node embeddings. It achieves an effective equilibrium across velocity as well as precision, rendering it advantageous for domains such as social networking sites, biological ones, and systems that offer recommendations.

Time complexity analysis

The ELP algorithm improves the classical LPA on a bottom-up basis. To obtain node embeddings, ELP handles a tensor decomposition problem after modeling the network as a tensor and projecting the nodes into a lower-dimensional embedding space. ELP and random walks are used to solve problems of instability, randomness, and resolution in classic LPA. Classical LPA runs in O (V + E) time complexity and depends only on the network topology. However, it oscillates in the labeling assigned to nodes and partitions the network in an unstable manner. ELP, on the other hand, employs node2vec and label propagation combined with embeddings to capture more structural and feature-based relations to achieve more stability and better detection quality.

Unlike modularity-based algorithms such as Louvain, which, like ELP, runs in O(VlogV) time, ELP shows the ability to detect more granular community structures, something modularity-based methods generally overlook because of the resolution limit. Infomap is better than ELP in that it can identify hierarchical communities through random walks, but it can struggle with dense graphs and produce communities that are irregularly shaped. ELP, on the other hand, improves the consistency of partitions formed as a result of random walk convergence through the embedding similarity approach. From a computation point of view, ELP adds an extra step during the generation of embeddings. Take, for instance, the semi-definite approximation of epigraphs. Its complexity is O(V^2 d), where V is the number of nodes and d is the embedding dimension. Each node is processed sequentially during the random walks for the node2vec embedding generation and matrix construction, yielding an overall embedding generation and matrix construction complexity that can be expressed as a linear function of V and E. While there is some overhead as compared to the linear time complexity associated with LPA, the increase in modularity, stability, and robustness makes the overhead worthwhile.

In conclusion, while algorithms like LPA, Louvain, Leiden, and Infomap are computationally efficient, their lack of stability, accuracy, and insensitivity to resolution of communities result in poor performance on complex networks. Community detection algorithms that utilize embeddings, like ELP, strike an ideal compromise between scalability and quality, which is why it is well suited for social, biological, and citation networks where relational semantics are important. Complex as it may be relative to traditional LPA, it is ELP's greater within-cluster density stability, more within-cluster density interpretability, and enhanced quality of the community overall that elevates it as more dependable when precise community identification is the main goal.

Result and discussion

Multiple real-world and artificially generated networks of different sizes were used for experiments. The objective of the experiments was to compare the quality and accuracy of the alternative methods of community detection that were evaluated with the aid of various measures. All the experiments were performed in Google Colab. For the experimental analysis, LPA [9] implementation provided by NetworkX library[32], PNLP[4], LBLD[10], PSN[33], Leiden[34], Louvain[35] and Infomap[36] were used.

To evaluate the count of communities detected with the original LPA and with Recent LPA Variants and ELP, dimensions such as detected communities, modularity, NMI, and NF1-Score were calculated for each of the networks, and then compared across all networks.

1. **Modularity(Q):** Modularity is a metric that shows how effectively a network is partitioned into modules. It examines how much interconnection there is within those groupings Compared to how much interconnection there is between groupings. The larger a network's modularity value (which falls between 0 and 1), the more pronounced the community structure.

$$Q = \frac{1}{2l} \sum_{ij} \left(M_{ij} - \frac{d_i d_j}{2l} \right) \delta(g_i, g_j) \tag{9}$$

where

- M_{ij}: Adjacency matrix of the network.
- d_i: Degree of node i.
- : Number of edges.
- $\delta(g_i, g_j)$: 1 if nodes i and j belong to the same community, 0 otherwise.

2. **Normalized Mutual Information (NMI)**: NMI is a way to check how similar two communities or groups are. It is useful when an actual community structure is available for comparison. It can be used to see how much the different data groups match up. This comparison can help understand whether community assignments catch the appropriate connections in the network. In short, NMI helps look at how closely different community structures align when networks with ground truth community structures are available. NMI ranges from 0 to 1, where 1 indicates perfect alignment with the ground truth communities, and 0 means no correlation.

Formula:

$$NMI(p, g) = \frac{2I(p; g)}{E(p) + E(g)} \tag{10}$$

where

- $I(p; g)$: Mutual information between the predicted communities p and the ground truth communities g.
- $E(p), E(g)$: Entropy of the predicted and ground truth communities.

3. **Number of Communities Detected**: Some time finding the count of number of communities detected in a network may give idea about the performance of the approach. But a great number of communities are not always a good idea, as well as a small one. To take an example, an extreme number of communities in a social network can be a sign of over-segmentation and insufficient number of communities can be a sign of under-segmentation. This number selected must be in line with the anticipated values. When the outcome does not resemble the expectation, then the data must be made to match the algorithm.

Let us understand. When the algorithm finds more than two or ten groups in a network while expecting approximately six, things go wrong. Too many groups might make the situation overly complicated, making it harder to understand the outcomes and less number of groups might cause details to be overlooked. Knowing the exact number of communities is important when it comes to analysis, decision-making, and outcomes. Consistency and correctness of the set of data are major.

4. **Normalized F1(NF1) score** Recall is described as the ratio of true positive vertices classified by the algorithm as belonging to a true community to the total number of vertices associated with that community. In essence, it assesses the extent to which an algorithm is successful in retrieving a set of group members. On the contrary, precision is the ratio of true positive vertices to the total number of vertices that an algorithm has classified as belonging to that community, reflecting the thoroughness of the estimator. A survey which aims to assess a trade-off of precision and recall would compute the F1 score, whch is the derived as the harmonic mean of the two. This score is representative of a community's relative measure of balance between the total and true number of detected community constituents.

Dataset	Algorithms							
	ELP	PNLP	Louvain	Infomap	PSN	LPA	Leiden	LBLD
ZKC	**0.46**	0.43	0.43	0.40	0.49	0.40	0.43	0.47
DCN	**0.43**	0.49	0.63	0.63	0.49	0.46	0.63	0.49
PolBooks	**0.64**	0.60	0.63	0.63	0.41	0.61	0.64	0.41
PolBlogs	**0.48**	0.43	0.44	0.10	0.10	0.48	0.10	0.49

Table 1. Modularity scores on four real benchmark networks. Significant values are in bold.

Dataset name	Actual communities	Algorithm							
		ELP	PNLP	Louvain	Infomap	PSN	LPA	Leiden	LBLD
ZKC	2	2	4	4	3	4	3	4	4
DCN	2	4	4	5	6	5	6	4	5
PolBooks	3	3	4	4	6	4	4	5	5
PolBlogs	2	8	11	10	42	3	12	12	11

Table 2. Total count of communities on four real-world networks dataset.

$$recall = \frac{|C_r \cap C_g|}{C_g} \tag{11}$$

$$precision = \frac{|C_r \cap C_g|}{C_r} \tag{12}$$

$$F1 - score = 2\frac{precision * recall}{precision + recall} \tag{13}$$

Let C_r represent the community identified by the method, whereas C_g denotes the actual community with whom the specified initiating point is affiliated. The F1-score scores vary from 0 to 1. Higher numbers indicate superior efficiency of the algorithm.

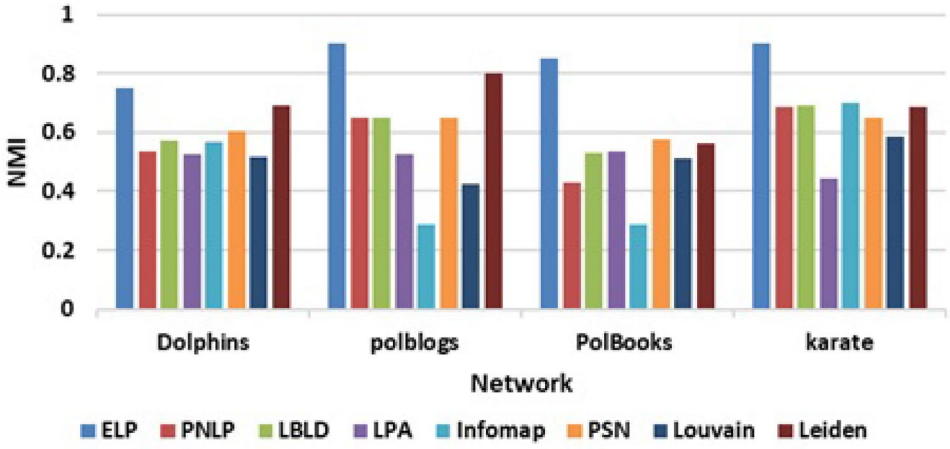
Experiments on real benchmark network

In the experiment, the real-benchmark networks highly used in the domain of social networks analysis were selected. In addition, each of these networks is offered at no cost with the intended use of conducting experiments. In the present investigation, four distinct networks are selected, such as the Zachary karate network(ZKN)[37], Dolphins network[38], Pol-books network[39] and Political blog network (PBN)[40]. The detail description is given as follows:

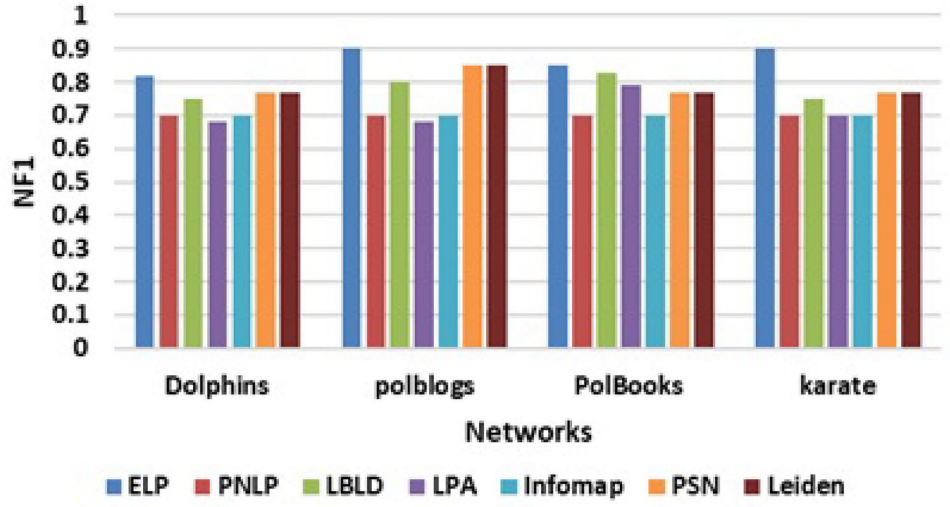
1. **Zachary Karate Network:** The network has 34 members, denoted by 34 nodes in graph representation, and 78 edges model the relationships between members. Wayne Zachary collected the data and modeled it in the 1970s as an undirected graph. Due to some dispute between two tutors at a karate club, two sections emerged.
2. **Polbooks Network:** It is a compilation of political writings aligned to the United States which came out in the year 2004. It can be procured from the online retail store Amazon.com. The interconnections among the volumes portray that the same consumers tend to get multiple copies of the same titles.
3. **Dolphins Network:** The dolphins' relationship constitutes a network of 159 edges and 62 vertices. 62 distinct dolphin species and interrelations of kinship were used to construct this network. Lusseau et al.[38] created a graph pertaining to this member's relations without direction.
4. **Political Blogs Network:** It is a structured, interlinked web. The interlinks demonstrate the connections between blogs. The network consists of two components: liberal and conservative. The entire blogsite network contains 1490 nodes and 19090 links.

Considering the Real Benchmark Network datasets, ELP retains high modularity relative to classical and contemporary approaches to LPA (see Table 1). The result given in Table 2 conclude that ELP can detect the number of communities closer to actual count. This is simply because ELP employs embeddings that capture both the local and global structural attributes of the network, thereby identifying finer and better-defined communities. ELP generates structurally stronger, more interpretable, and more consistent with the natural nature of the network communities by using the information of the overall topology, instead of only localized connections. Simply put, the algorithm finds groups that do not only make sense locally, but also retain the network dynamics at large scale, creating communities that are more significant and coherent.

The increased performance of ELP can also be seen through its NMI and NF1 (see Fig. 2) scores in which it is always higher than the original LPA and its new extensions. This indicates that ELP is more representative of the actual community structure in that it effectively incorporates localized connectional patterns with global



(a) Normalized Mutual Information Score



(b) Normalized F1-Score

Fig. 2. Analyzing the output of various techniques to determine the presence of a community structure within real-world networks.

structural clues. By synchronizing these various levels of information ELP gives a better idea of relationships between nodes in the network.

In addition to such performance benefits, there are the added benefits of ELP. The embedding-based design is more stable in the detecting of the community, firstly, because the method is less sensitive to random initialization and stochastic dynamics that tend to restrict the traditional LPA methods. Second, the reported modularity benefits imply that ELP can identify more coherent and clean communities and this is especially relevant to systems that need to trust the community structure to be well-defined, like the analysis of biological networks or recommendation systems. Lastly, the embedded-based structure of ELP also makes it very appropriate to large, and dynamically changing networks where scalability and adaptability are paramount. With network systems becoming increasingly more complex and dynamic, the capacity of ELP to stay stable, attain modular partitions, and scale is an important indicator to note that ELP is capable of being a powerful and highly adaptable instrument in the real-world community detection process.

Experiments on artificial benchmark networks

During the experiments, multiple different networks are constructed using the Lancichinetti-Fortunato-Radicchi (LFR) model[41]. Constructing artificial benchmark networks requires the input of different parameters, which are described in Table 3. The Table 4 describe the different values used for the generation of nine artificial networks.

LFR benchmark networks

As a means of construct computer networks, the researchers used the LFR algorithm as a baseline in order to assess the outcome of community identification. The Academic community's use of LFR benchmarks is mostly due to its ability to construct networks whose configurations bear a close resemblance to actual benchmark

Parameter	Description
N	Nodes in the system
k	Mean number of connections
maxk	Highest number of connections
mu	Fraction of edges within other communities
t1	Exponent determining the distribution of the degrees
t2	Exponent determining the distribution of the community sizes
minc	Limit to number of members in a community
maxc	Upper limit to number of members in a community
on	Nodes shared by more than one community
om	Total number of communities associated with each overlapping node

Table 3. Parameters in LFR-NW benchmark.

Parameter	LFR-NW1	LFR-NW2	LFR-NW3	LFR-NW4	LFR-NW5	LFR-NW6	LFR-NW7	LFR-NW8	LFR-NW9
N	1000	5000	5000	10000	10,000	20,000	50,000	100,000	300,000
k	15	20	20	20	20	15	15	15	15
k_{max}	50	50	20	100	100	50	50	50	50
μ	0.2	0.1	0.1	0.2	0.2	0.4	0.1	0.1	0.1
e_d	2	2	2	2	2	2	2	2	2
e_c	1	1	2	1	2	1	1	1	1
C_{min}	10	20	20	20	20	20	20	20	20
C_{max}	50	100	100	1000	1000	50	50	50	50

Table 4. Parameter configurations for the generation of artificial networks using the LFR algorithm.

Dataset	Algorithms							
	ELP	PNLP	Louvain	Infomap	PSN	LPA	Leiden	LBLD
LFR-NW1	**0.76**	0.71	0.76	0.76	0.42	0.74	0.76	0.74
LFR-NW2	**0.88**	0.86	0.88	0.88	0.85	0.88	0.88	0.83
LFR-NW3	**0.89**	0.87	0.88	0.88	0.86	0.88	0.88	0.85
LFR-NW4	**0.76**	0.70	0.75	0.75	0.25	0.75	0.75	0.75
LFR-NW5	**0.76**	0.72	0.76	0.76	0.29	0.76	0.76	0.75
LFR-NW6	**0.60**	0.51	0.60	0.59	0.31	0.59	0.60	0.66
LFR-NW7	**0.90**	0.89	0.90	0.59	0.54	0.89	0.90	0.91
LFR-NW8	**0.90**	0.89	0.90	0.89	0.88	0.89	0.95	0.94
LFR-NW9	**0.90**	0.88	0.90	0.89	0.87	0.89	0.90	0.95

Table 5. The value of modularity scores achieved for the structures of communities detection by ELP and other well-known algorithms on networks generated by the LFR algorithm. Significant values are in bold.

networks. Furthermore, followed in the networks created by the LFR benchmark are real-world network properties, including power-law distribution in both the nodes' degrees and the communities' sizes. More precisely, below are these traits. Consequently, the LFR benchmark is selected to be the benchmark against which algorithms are assessed.

At the same time, nine distinct synthetic benchmarking networks are generated for the purposes of this experiment, each differing in shape and features. Every one of these networks contains various defining characteristics such as the vertex count, average connectivity, and the configuration of the mixing matrix together with the sequence of community sizes and the configuration of the sizes of the communities in a network; the minimum, maximum, and the dispersed community clusters with a network. These kinds of networks are referred to by their abbreviations, which range from LFR-NW1 to LFR-NW9. Table 4 provides further information regarding the values that were utilized in the process of generating these networks.

The values of modularity achieved by the ELP, PSN, Infomap, LPA, Louvain and Leiden were compared to assess the resultant community structures detected by these algorithms. From the result presented in Table 5, it is clear that proposed algorithm ELP and other five well-known algorithms (i.e. Louvain, PSN, Infomap, LPA and Leiden) were able to achieve the good result.

Dataset	#comm.	Algorithms							
		ELP	PNLP	Louvain	Infomap	PSN	LPA	Leiden	LBLD
LFR-NW1	36	**35**	29	34	36	35	26	34	33
LFR-NW2	101	**98**	81	97	101	100	92	98	97
LFR-NW3	130	**123**	101	110	130	123	118	109	115
LFR-NW4	45	**45**	41	43	45	45	41	43	44
LFR-NW5	126	**125**	121	83	126	125	105	81	120
LFR-NW6	831	**815**	799	132	831	417	827	114	810
LFR-NW7	1537	**1520**	1487	381	414	512	1537	355	1530
LFR-NW8	3057	**3017**	3010	498	3057	758	3056	484	3020
LFR-NW9	9240	**9214**	9045	817	9240	1470	9230	799	9004

Table 6. The count of communities identified by the ELP along with some other benchmark algorithms on LFR synthesised networks. Significant values are in bold.

In comparison to other approaches, ELP has successfully pinpointed a considerable amount of communities in all LFR generated networks, as illustrated in Table 6 that elaborates on different communities identified by ELP, Louvain, Infomap, PSN, LPA, and Leiden algorithms.

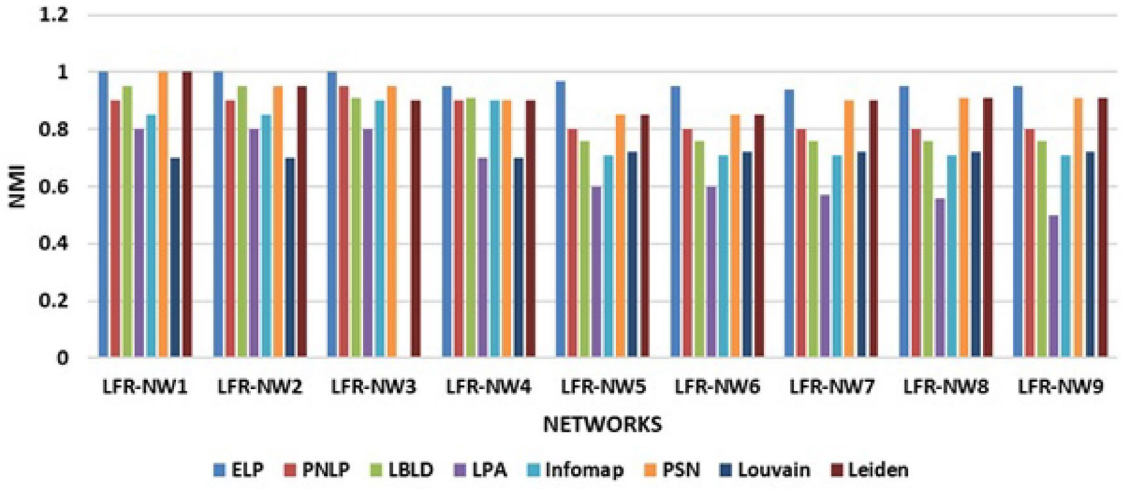
NMI scores attained by the ELP and other algorithms are presented in Fig. 3a . The suggested algorithm (ELP), Louvain, LPA, Infomap, and Leiden reached an NMI score of 1 in experiments on the LFR-NW1, LFR-NW2, and LFR-NW3 networks. The conclusion was reached that these algorithms worked extraordinarily well over the varying types of networks. This also indicates that ELP can intelligently manage networks of different types and sizes while maintaining a respectable score.

Figure 3b displays the F1 score calculated for the networks generated using the LFR algorithm. ELP algorithm successfully achieved a higher F1 score than the Louvain, Leading Eigenvector, and Leiden algorithms. LPA and Infomap have the potential to achieve satisfactory results for LFR-NW1 to LFR-NW3 networks, respectively. The LPA method performed well for all LFR-generated networks. The findings of the study indicate that the newly developed strategy has the potential to detect social structures in a range of medium to large networks more accurately than many other established strategies.

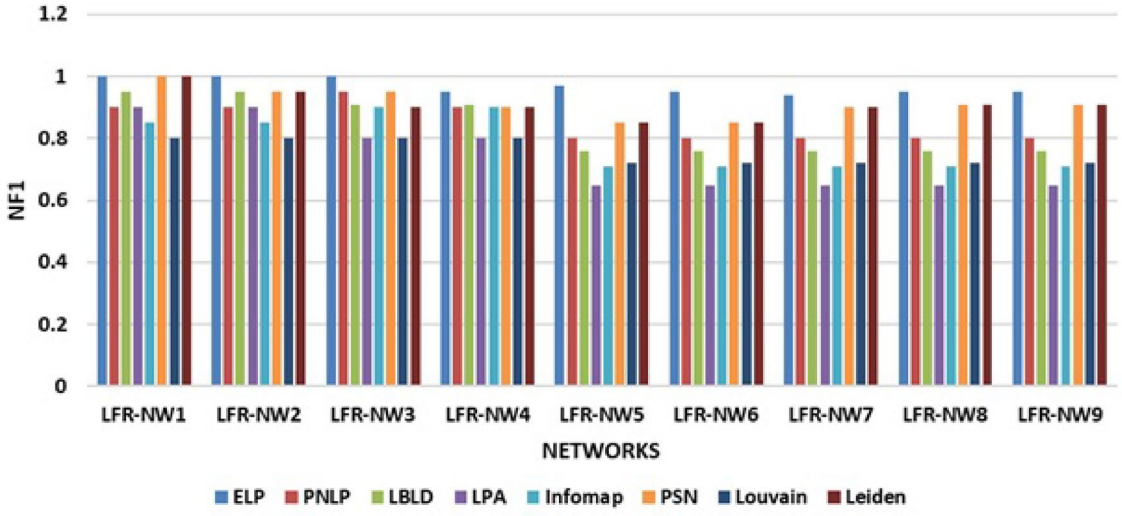
Conclusion

The study proposes a novel algorithm called Embedding-based Label Propagation which optimally improves community. ELP uses local neighborhood and global structure data to overcome the limitations of traditional LPA and produces more prominent, stable, and coherent communities across different types of networks. Performance evaluation on synthetic and real-world benchmark data shows ELP improves modularity, accuracy and robustness compared to traditional algorithms but retains the scalability and simplicity of LPA. ELP retains the same low complexity as label propagation while providing more certainty on large complex networks due to reduced randomness.

We recognize that embedding computing nodes comes with extra overhead, thereby making ELP slower than the original LPA. The next line of work to be undertaken in future is to streamline embedding computation and to consider parallel or distributed methods of processing to drive scaling and efficiency even higher. The exciting future potential would be to scale ELP to dynamic and overlapping community detection which will increase its applicability to real-world networks that change through time and have multiplexing structures.



(a) NMI scores



(b) NF1 scores

Fig. 3. Comparison of Normalized Mutual Information (NMI) and Normalized F1 (NF1) scores achieved on LFR-generated Benchmark networks.

Data availability

The datasets used and/or analysed during the current study will be available from the corresponding author on reasonable request.

Received: 30 July 2025; Accepted: 24 October 2025
Published online: 25 November 2025

References

1. Satuluri, V. *et al.* Simclusters: Community-based representations for heterogeneous recommendations at twitter. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 3183–3193 (2020).
2. Manipur, I., Giordano, M., Piccirillo, M., Parashuraman, S. & Maddalena, L. Community detection in protein-protein interaction networks and applications. *IEEE/ACM Trans. Comput. Biol. Bioinform.* **20**, 217–237 (2021).
3. Jin, D. et al. A survey of community detection approaches: From statistical modeling to deep learning. *IEEE Trans. Knowl. Data Eng.* **35**, 1149–1170 (2021).
4. Guo, P., Tang, Q. & Guo, J. Community detection using an improved label propagation algorithm. In *2024 IEEE International Conference on Big Data (BigData)*, 7031–7040 (IEEE, 2024).
5. Kumar, R., Behera, R.K., Kesarwani, A. & Rath, S.K. A fast algorithm for enumerating maximal cliques in large scale network. In *2017 14th IEEE India Council International Conference (INDICON)*, 1–6, https://doi.org/10.1109/INDICON.2017.8488050 (2017).
6. Sismanis, K., Potikas, P., Souliou, D. & Pagourtzis, A. Overlapping community detection using graph attention networks. *Future Generat. Comput. Syst.* **163**, 107529 (2025).
7. Papadopoulos, N.A. & Tzortzidis, G. Distributed time-based local community detection. In *Proceedings of the 24th Pan-Hellenic Conference on Informatics*, 390–393 (2020).

8. Shang, R., Zhang, W., Zhang, J., Feng, J. & Jiao, L. Local community detection based on higher-order structure and edge information. *Physica A Stat. Mech. Appl.* **587**, 126513 (2022).
9. Raghavan, U. N., Albert, R. & Kumara, S. Near linear time algorithm to detect community structures in large-scale networks. *Phys. Rev. E* **76**, 036106. https://doi.org/10.1103/PhysRevE.76.036106 (2007).
10. Roghani, H. & Bouyer, A. A fast local balanced label diffusion algorithm for community detection in social networks. *IEEE Trans. Knowl. Data Eng.* **35**, 5472–5484 (2022).
11. Li, P.-Z., Huang, L., Wang, C.-D., Lai, J.-H. & Huang, D. Community detection by motif-aware label propagation. *ACM Trans. Knowl. Discov. Data (TKDD)* **14**, 1–19 (2020).
12. Zarezadeh, M., Nourani, E. & Bouyer, A. Dpnlp: distance based peripheral nodes label propagation algorithm for community detection in social networks. *World Wide Web* 1–26 (2022).
13. Lu, M., Zhang, Z., Qu, Z. & Kang, Y. Lpanni: Overlapping community detection using label propagation in large-scale complex networks. *IEEE Trans. Knowl. Data Eng.* **31**, 1736–1749 (2018).
14. Bouyer, A. & Roghani, H. Lsmd: A fast and robust local community detection starting from low degree nodes in social networks. *Future Generation Comput. Syst.* **113**, 41–57 (2020).
15. Zhang, X.-K., Ren, J., Song, C., Jia, J. & Zhang, Q. Label propagation algorithm for community detection based on node importance and label influence. *Phys. Lett. A* **381**, 2691–2698 (2017).
16. Aghaalizadeh, S., Afshord, S. T., Bouyer, A. & Anari, B. A three-stage algorithm for local community detection based on the high node importance ranking in social networks. *Physica A Stat. Mech. Appl.* **563**, 125420 (2021).
17. Kong, H. et al. An improved label propagation algorithm based on node intimacy for community detection in networks. *Int. J. Modern Phys. B* **32**, 1850279 (2018).
18. Yue, Y., Wang, G., Hu, J. & Li, Y. An improved label propagation algorithm based on community core node and label importance for community detection in sparse network. *Appl. Intell.* **53**, 17935–17951 (2023).
19. Kaixuan, D., Hongchang, C. & Ruiyang, H. Improved lpa algorithm based on label propagation ability. *Comput. Eng.* **44**, 60–64 (2018).
20. Zarezade, M., Nourani, E. & Bouyer, A. Community detection using a new node scoring and synchronous label updating of boundary nodes in social networks. *J. AI Data Mining* **8**, 201–212 (2020).
21. Thakare, S.B. & Kiwelekar, A.W. Skiplpa: An efficient label propagation algorithm for community detection in sparse network. In *Proceedings of the 9th Annual ACM India Conference*, 97–106 (2016).
22. Yuan, H. et al. A community discovery method based on core node influence [j]. *Comput. Sci.* **49**, 240–246 (2022).
23. Li, H., Zhang, R., Zhao, Z. & Liu, X. Lpa-mni: An improved label propagation algorithm based on modularity and node importance for community detection. *Entropy* **23**, 497 (2021).
24. Lin, Z., Zheng, X., Xin, N. & Chen, D. Ck-lpa: Efficient community detection algorithm based on label propagation with community kernel. *Physica A Stat. Mech. Appl.* **416**, 386–399 (2014).
25. Zhao, X., Liang, J. & Wang, J. A community detection algorithm based on graph compression for large-scale social networks. *Inform. Sci.* **551**, 358–372 (2021).
26. Zhang, W., Shang, R. & Jiao, L. Large-scale community detection based on core node and layer-by-layer label propagation. *Inform. Sci.* **632**, 1–18 (2023).
27. Zhenxin, Z., Yuecheng, Y., Guyu & Daoyue, J. Label propagation community discovery method based on layer-by-layer expansion of core nodes. *Comput. Digital Eng.* **50**, 1327–1333 (2022).
28. Fei, R. et al. Deep core node information embedding on networks with missing edges for community detection. *Inform. Sci.* **707**, 122039 (2025).
29. Shen, C. *et al.* Principal graph encoder embedding and principal community detection. *IEEE Trans. Netw. Sci. Eng.* (2025).
30. Kumar, J. S., Archana, B., Muralidharan, K. & Kumar, V. S. Graph theory: Modelling and analyzing complex system. *Metallurgical Mater. Eng.* **31**, 70–77 (2025).
31. Grover, A. & Leskovec, J. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, 855–864 (2016).
32. Hagberg, A.A., Schult, D.A. & Swart, P.J. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*, 11–15 (Pasadena, CA USA, 2008).
33. Meena, S.S. & Tokekar, D.V. Community detection in social networks by incorporating the preferential selection. *J. Complex Netw.* **9**, cnab040 (2021).
34. Traag, V. A., Waltman, L. & Van Eck, N. J. From louvain to leiden: Guaranteeing well-connected communities. *Sci. Rep.* **9**, 1–12 (2019).
35. Blondel, V. D., Guillaume, J.-L., Lambiotte, R. & Lefebvre, E. Fast unfolding of communities in large networks. *J. Stat. Mech. Theory Exp.* **2008**, P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008 (2008).
36. Rosvall, M., Axelsson, D. & Bergstrom, C. T. The map equation. *Eur. Phys. J. Special Topics* **178**, 13–23 (2009).
37. Zachary, W. W. An information flow model for conflict and fission in small groups. *J. Anthropol. Res.* **33**, 452–473 (1977).
38. Lusseau, D. et al. The bottlenose dolphin community of doubtful sound features a large proportion of long-lasting associations. *Behav. Ecol. Sociobiol.* **54**, 396–405 (2003).
39. Newman, M. E. Modularity and community structure in networks. *Proc. Natl. Acad. Sci.* **103**, 8577–8582 (2006).
40. Adamic, L.A. & Glance, N. The political blogosphere and the 2004 us election: divided they blog. In *Proceedings of the 3rd international workshop on Link discovery*, 36–43 (2005).
41. Lancichinetti, A., Fortunato, S. & Radicchi, F. Benchmark graphs for testing community detection algorithms. *Phys. Rev. E* **78**, 046110 (2008).

Author contributions

Shyam Sundar Meena: Writing - original draft, Visualization, Software, Methodology, Investigation, Formal analysis, data curation, Conceptualization. Mukund Pratap Singh: Visualization, Formal analysis. Prakash Chandra Sharma: Writing - review & editing, Supervision, Resources, Methodology, Investigation, Conceptualization, Correspondence. Yadvendra Pratap Singh: Formal analysis,Review, Investigation, Correspondence, Native English language verification.

Funding

Open access funding provided by Manipal University Jaipur. This work was carried out without any financial support from any organization.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to P.C.S. or Y.P.S.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025